# 🔷 Wooden Block Shape Recognition and Sorting

## 1. Course Overview

The function implemented in this lesson can be simply understood as: **camera distinguishes shapes, judges whether it's a square or cylinder based on contours, automatically places them in corresponding slots**.

The program first uses mouse box selection for color, locking onto the target block's HSV color range. Then for each frame, it performs color segmentation to extract contours, using **polygon fitting vertex count + aspect ratio + circularity** three geometric features combined to determine shape (square/rectangle/cylinder/triangle), and uses multi-frame voting mechanism to stabilize shape labels. Finally, the robotic arm automatically sorts and unloads based on shape.

> **Note**
>
> **The tutorial recommends using the accompanying 3×3cm colored wooden blocks. The program supports recognition of square, rectangular, cylindrical, and triangular wooden blocks.**

---

## 2. Preparation and Running

This case requires opening 3 terminals to start the system sequentially:

### 2.1 Start Camera Driver and Kinematics Solver Environment

Run in **Terminal 1**:

```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```

```
jetson@yahboom:~$ ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
[INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-09-02-16-32-09-195196-yahboom-1117638
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [camera_launch-1]: process started with pid [1117670]
[INFO] [kinemarics_dofbot-2]: process started with pid [1117672]
[INFO] [Ackman_driver_A1P-3]: process started with pid [1117674]
[kinemarics_dofbot-2] -----------------
[kinemarics_dofbot-2] fk
[kinemarics_dofbot-2] 5
[kinemarics_dofbot-2] -----------------
[camera_launch-1] [INFO] [launch]: All log files can be found below /home/jetson/.ros/log/2026-09-02-16-32-10-139043-yahboom-1117670
[camera_launch-1] [INFO] [launch]: Default logging verbosity is set to INFO
[camera_launch-1] [INFO] [usb_cam_node_exe-1]: process started with pid [1117735]
[camera_launch-1] [usb_cam_node_exe-1] libEGL warning: DRI2: failed to authenticate
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337930.852521184] [usb_cam]: camera_name value: test_camera
[camera_launch-1] [usb_cam_node_exe-1] [WARN] [1788337930.852749618] [usb_cam]: framerate: 120.000000
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337930.876670665] [usb_cam]: camera calibration URL: package://usb_cam/config/cam
era_info.yaml
[camera_launch-1] [usb_cam_node_exe-1] [WARN] [1788337930.879330849] [usb_cam]: [test_camera] does not match narrow_stereo in file /
home/jetson/yahboomcar_ros2_ws/software/library_ws/install/usb_cam/share/usb_cam/config/camera_info.yaml
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337930.953127689] [usb_cam]: Starting 'test_camera' (/dev/video0) at 640x480 via
mmap (mjpeg2rgb) at 120 FPS
[camera_launch-1] [usb_cam_node_exe-1] [swscaler @ 0xaaaad038ffe0] No accelerated colorspace conversion found from yuv422p to rgb24.
[camera_launch-1] [usb_cam_node_exe-1] This device supports the following formats:
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1280 x 720 (60 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1920 x 1080 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1024 x 768 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 640 x 480 (120 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 800 x 600 (60 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 1280 x 1024 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  Motion-JPEG 320 x 240 (120 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1280 x 720 (9 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1920 x 1080 (6 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1024 x 768 (6 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 640 x 480 (30 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 800 x 600 (20 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 1280 x 1024 (6 Hz)
[camera_launch-1] [usb_cam_node_exe-1]  YUYV 4:2:2 320 x 240 (30 Hz)
[Ackman_driver_A1P-3] [INFO] [1788337931.035307899] [driver_node]: A1 driver initialized: imu_link=imu_link, pub_hz=20.0, arm_status
_hz=5.0, cmd_vel_timeout=0.50s, pub_odom=True
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.748886319] [usb_cam]: Setting 'white_balance_temperature_auto' to 1
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.749045211] [usb_cam]: Setting 'exposure_auto' to 3
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.760676716] [usb_cam]: Setting 'focus_auto' to 0
[camera_launch-1] [usb_cam_node_exe-1] [INFO] [1788337931.836160385] [usb_cam]: Timer triggering every 8 ms
```

## 2.2 Start Robotic Arm Desktop Grasping Action Execution Node

Run in **Terminal 2**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver grasp_desktop

# Or USB camera version
ros2 run dofbot_pro_usb grasp_usb
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb grasp_usb
{'x_offset': 0.009368814795573138, 'y_offset': 0.007718452024610358, 'z_offset': 0.0}
----------------------------
x_offset:  0.009368814795573138
y_offset:  0.007718452024610358
z_offset:  0.0
----------------------
[INFO] [1788337928.183721749] [Grasp_Node]: Service not available, waiting again...
[INFO] [1788337929.186574716] [Grasp_Node]: Service not available, waiting again...
init done
self.CurEndPose:  [0.0938259179959246, 0.0002628454756523636, 0.12303978979136244, -4.489092034631593e-06, 1.3962632447969856, -2.82
89145258287586e-05]
```

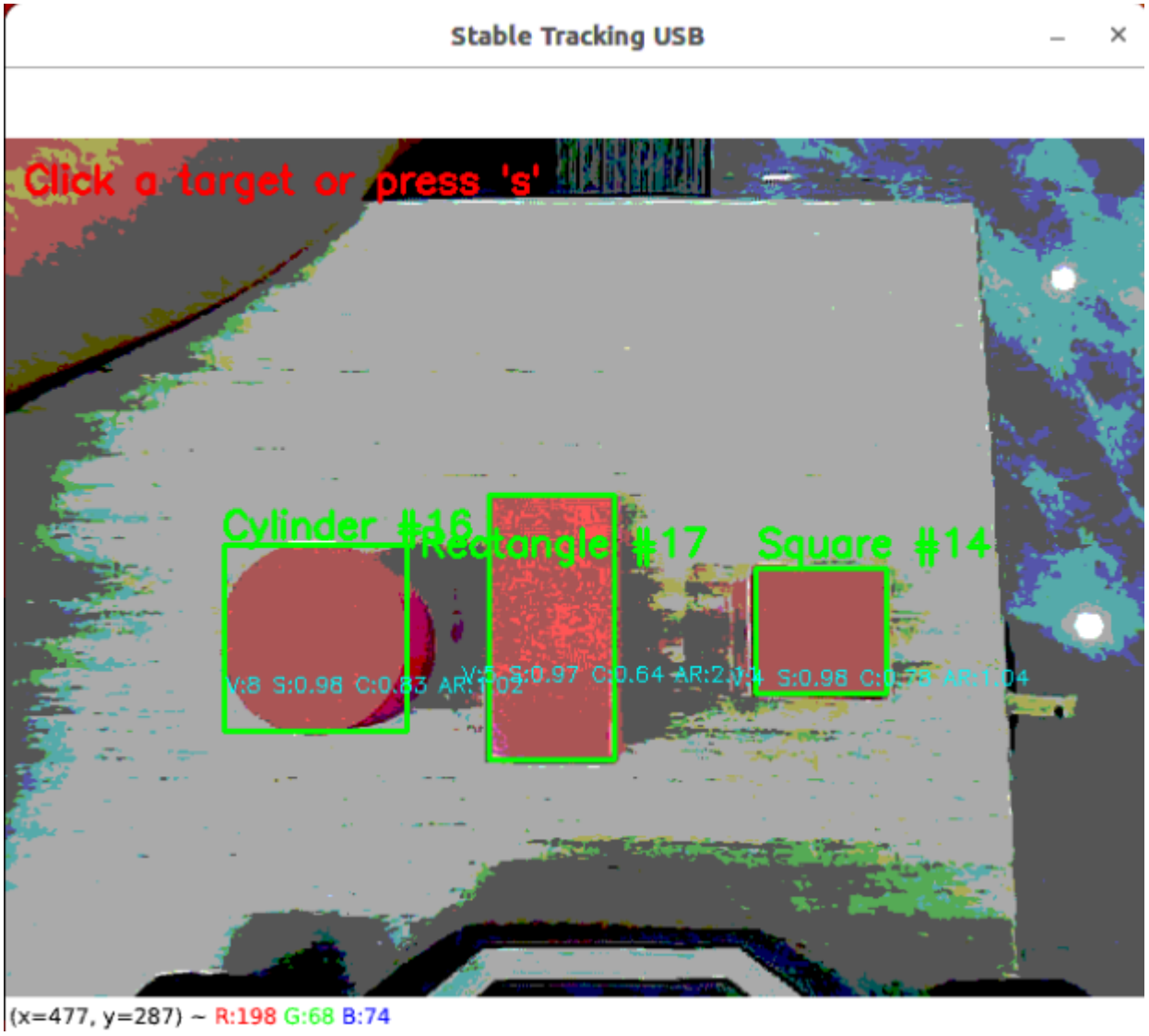
## 2.3 Start Geometric Shape Recognition and Sorting Master Node

Run in **Terminal 3**:

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver shape_recognize

# Or USB camera version
ros2 run dofbot_pro_usb shape_usb_recognize
```

```
jetson@yahboom:~$ ros2 run dofbot_pro_usb shape_usb_recognize
[INFO] [1788337973.571396269] [ShapeUSBRecognize_node]: Stable anti-flicker wood block recognition
started!
[INFO] [1788337973.911884558] [ShapeUSBRecognize_node]: Robotic arm driver connected, initial pose
sent.
```




## 3. Operation Demo and Sorting Interaction

1. Place square, rectangular, or circular wooden blocks steadily below the robotic arm field of view;
2. Press `C` to enter color selection mode **(Note: depth camera version does not require color selection)**, use mouse to drag and select the target wooden block area to sample HSV. The image will display real-time detection and annotation of each block's geometric shape name ( `Square` / `Rectangle` / `Cylinder` / `Triangle` );
3. **Left-click** on the specific target block to grasp. The selected block's border will change to highlighted 🔴 red;
4. Press **spacebar or enter ( `Space` / `Enter` )**, chassis automatically and smoothly aligns, robotic arm descends to grasp the target block and places it in the fixed slot.

> **Tip**

> If environmental lighting changes causing incomplete contour extraction, you can press `C` at any time to re-select the wooden block for color sampling.

---

# 4. Core Source Code Analysis

The system is completed by "geometric shape recognition decision" and "IK grasping execution" two nodes working in coordination:

| Camera Type | Shape Recognition and Selection Master Node | Grasping Action Execution Node |
| --- | --- | --- |
| **USB Monocular Camera** | `dofbot_pro_usb/shape_usb_recognize.py` | `dofbot_pro_usb/grasp_usb.py` |
| **3D Depth Camera** | `dofbot_pro_driver/shape_recognize.py` | `dofbot_pro_driver/grasp_desktop.py` |

The core business logic of both remains unified:
`HSV Segmentation -> Extract Geometric Features (vertices/aspect ratio/circularity) -> Historical queue multi-frame voting lock -> Mouse click to select target -> Calculate 3D coordinates and gripper angle -> IK execution bin grasping`.

The main technical implementation comparison between the two:

| Comparison Dimension | USB Monocular Camera Solution | 3D Depth Camera Solution (Nuwa-HP60C) |
| --- | --- | --- |
| **3D Depth Calculation** | Calculate spatial coordinates through pinhole model referencing table baseline height | Hardware-level depth map directly indexes true millimeter distance at selected block centroid |
| **Hand-Eye Spatial Compensation** | Superimpose X, Y 2D spatial calibration deviation | Synchronously superimpose X, Y, Z 3D omnidirectional calibration compensation |
| **Shape Fitting Features** | Combine polygon vertex count, aspect ratio, compactness, and circularity for multi-dimensional discrimination | Combine polygon fitting vertex count and edge length tolerance to discriminate square, rectangle, and cylinder |
| **Stability Filtering** | 10-frame historical queue with same shape appearing 6 times to lock label | Similar sliding window historical queue voting mechanism, preventing label single-frame flicker and jumping |

---

## 4.1 USB Monocular Camera Core Source Code Analysis (`shape_usb_recognize.py`)

### 1. Geometric Feature Combined Recognition Algorithm

After extracting contours, the program calculates circularity (`circularity`), aspect ratio (`aspect_ratio`), and polygon fitting vertex count (`vertices`), using four-dimensional feature joint arbitration:

```
# shape_usb_recognize.py —— Geometric joint discrimination
perimeter = cv2.arcLength(obj, True)
circularity = 4 * math.pi * area / (perimeter * perimeter)
aspect_ratio = max(rect_w, rect_h) / min(rect_w, rect_h)
approx = cv2.approxPolyDP(obj, 0.035 * perimeter, True)
vertices = len(approx)

if circularity > 0.78 and vertices > 6:
    obj_type = "Cylinder"     # High circularity + multi-vertex fitting -> Cylinder
elif 4 <= vertices <= 6:
    if aspect_ratio < 1.15:
        obj_type = "Square"   # Vertices 4-6 and aspect ratio close to 1 -> Cube
    else:
        obj_type = "Rectangle" # Clearly elongated -> Rectangular prism
elif vertices == 3:
    obj_type = "Triangle"     # 3 vertices -> Triangle
```

### 2. Multi-Frame Voting and Mouse Polygon Collision Detection

Maintains a historical queue with capacity of 10; the most frequent shape needs to accumulate 6 occurrences before locking display; when mouse clicking, uses `cv2.pointPolygonTest` to determine whether the click point is within the polygon bounding box:

```
# shape_usb_recognize.py —— Mouse click detection
for t_obj in self.tracked_objects:
    if cv2.pointPolygonTest(t_obj['box'], (float(x), float(y)), False) >= 0:
        clicked_targets.append((distance, t_obj))

# Lock selected target ID and change border to red
self.selected_track_id = selected['track_id']
self.Target_Shape = selected['locked_shape']
```

---

## 4.2 3D Depth Camera Core Source Code Analysis (`shape_recognize.py`)

### 1. Depth Map Direct Extraction and World Absolute Coordinate Solving

The depth camera solution obtains aligned point cloud depth matrix through `ApproximateTimeSynchronizer`. When the user mouse-selects a target, it directly reads the true distance at the block center pixel and calculates the robotic arm base world coordinates:

```
# shape_recognize.py —— Target depth extraction and hand-eye transform
cx = int(target['center'][0])
cy = int(target['center'][1])
# Directly read true depth (meters)
dist = depth_image[cy, cx] / 1000.0

# Combine hand-eye matrix and robotic arm current forward kinematics to compute world
coordinates pose
pose = self.compute_heigh(cx, cy, dist)
# Superimpose offset_value.yaml 3D spatial calibration compensation
pose[0] += self.x_offset
pose[1] += self.y_offset
pose[2] += self.z_offset


   # 50 is the grayscale threshold, used to remove darker areas: grayscale range is 0~255,
this sets pixels below 50 to black, keeping the rest. When doing subsequent binarization and
contour recognition, areas set to zero usually won't be treated as targets
   # Increasing 50: filters more areas, may reduce interference but may also filter out
objects.
   # Decreasing 50: keeps more areas, but may increase noise and extra contours.
    black_image = np.zeros_like(gray_image)
    black_image[0:320, 0:640] = gray_image[0:320, 0:640]
    black_image[black_image < 50] = 0
```

## 2. Gripper Parallel Alignment Rotation Angle Calculation

Using the minimum bounding rectangle two vertex difference vectors from the selected block's contour fitting, real-time solve joint 5 pose angle to ensure the gripper closes parallel to the square or rectangular block's side wall:

```
# shape_recognize.py —— Gripper adaptive deflection angle computation
vx = target['box'][0][0] - target['box'][1][0]
vy = target['box'][0][1] - target['box'][1][1]
target_joint5 = compute_joint5(vx, vy)
self.joint5.data = int(target_joint5)

# Publish adaptive deflection angle and 3D pose
self.TargetJoint5_pub.publish(self.joint5)
self.pos_info_pub.publish(pos)
```

## 3. Shape Classification Placement

After the grasping execution node receives target information, it maps to different joint rotation target angles based on the recognized geometric shape (`Target_Shape`), driving rotation to different unloading stations for cubes, rectangular prisms, and cylinders, completing the entire shape sorting automation closed-loop.

## 4.3 Grasping Offset Tuning (Both Camera Models)

If position deviation occurs during actual grasping (such as missing, knocking over objects, offset left/right, or hitting the table), you can directly modify the spatial offset configuration file for the corresponding camera:

### 1. Where to Modify

Configuration files are located in the `~/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_driver/config/` directory. Select based on camera type:

- **3D Depth Camera (Nuwa-HP60C)**: `offset_value.yaml`
- **USB Camera**: `usb_offset_value.yaml`

### 2. How to Modify

The file contains offset parameters in X, Y, Z directions (unit: meters, recommended adjustment per tweak is `0.005`~`0.01`, i.e., 5~10mm):

- `x_offset` **(forward/backward direction, positive toward vehicle front)**:
  - Gripper extends too far and knocks over object **decrease value** (adjust smaller or set negative)
  - Gripper doesn't extend far enough and misses **increase value** (adjust larger)
- `y_offset` **(left/right direction, positive toward vehicle left)**:
  - Gripper deviates right **increase value** (correct left)
  - Gripper deviates left **decrease value** (correct right)
- `z_offset` **(up/down height, positive toward ceiling)**:
  - Gripper too high in air and can't grasp **decrease value** (probe deeper)
  - Gripper hits or scrapes table **increase value** (raise higher)